

Networking Technologies and Applications

Cours et méthodes de calcul, séances 1 à 3

BMEVITMAC09, automne 2026. Rédigé le 2026-09-28.

À retenir

Midterm (ZH)	fin octobre, noté réussi ou échoué; date exacte À VÉRIFIER
Seuil	40 % selon la fiche 2023 ; seuil 2026 À VÉRIFIER
Rattrapage	dernière semaine du semestre (du 2026-12-07 au 2026-12-11), jour À VÉRIFIER
Examen final	période de session; date, format, documents autorisés À VÉRIFIER
Note finale	pondération midterm et examen À VÉRIFIER
Présence	séance pratique du jeudi obligatoire : 70 %, 4 absences au plus

Source : fiche CLAUDE.md de la matière; hta1-intro.pdf, p. 3

Piège

Le midterm est annoncé « end of October » et le voyage à Wuhan dure du 2026-10-27 au 2026-11-03 : collision possible tant que la date n'est pas connue. Demander à mate@tmit.bme.hu.

Ce que les slides disent de l'examen. Les diapositives « Standardization Bodies » (hta1p-protocols.pdf, p. 9 à 11) et « ASN.1 Example » (p. 23) sont marquées « Not exam material ». La p. 26 annonce une question possible : « turn the given XML into JSON/YAML ». Ce document signale ces pages et ne les développe pas.

Usage. Un concept par page ; les concepts 6 à 17 sont les méthodes de calcul, à refaire à la main. Exercices dans « NET, exercices corrigés ».

Sources. Chaque section cite sa slide (fichier, page du PDF). Les méthodes de calcul sont du savoir standard, signalé « méthode générale ». Tous les exemples chiffrés sont recalculés avec le module Python `ipaddress`. L'incertain est marqué **À VÉRIFIER**. Tableaux binaires : **bleu** pour le préfixe, **ocre** pour la partie hôte, | à la frontière.

Table des matières

1	Télécommunication, partage du support, adressage	5
1.1	Partager un support commun (p. 16 à 21)	5
1.2	Modes de communication et adressage (p. 22 à 25)	6
2	Protocoles, pile, modèles OSI et TCP/IP	7
2.1	Standardisation (p. 7 et 8)	8
3	Sécurité et qualité de service	9
3.1	Sécurité (p. 12 à 15)	9
3.2	QoS (p. 16 à 19)	9
4	Multiplexage statistique	10
5	Sérialisation : TLV, XML, JSON, YAML	11
5.1	Méthode : convertir du XML en JSON puis en YAML (méthode générale)	12
6	Binaire, décimal, hexadécimal	13
6.1	Décimal vers binaire, méthode des poids (soustractions) . . .	13
6.2	Décimal vers binaire, divisions successives par 2	13
6.3	Binaire vers décimal	14
6.4	Hexadécimal	14
7	Adresse IPv4 et notation pointée	16
8	Classes historiques	17
8.1	Pourquoi les classes ont été abandonnées (p. 5)	17
9	Adresses spéciales et privées	18
10	Masque de sous-réseau et notation CIDR	19

10.1 /n vers masque	19
10.2 Masque vers /n	19
10.3 Taille d'un réseau	20
11 Réseau, broadcast, plage d'hôtes : méthode du ET	21
12 Méthode rapide du nombre magique	23
13 Appartenance d'une adresse à un réseau	24
14 Découpage en sous-réseaux de taille fixe	25
15 VLSM : sous-réseaux de tailles différentes	27
16 Agrégation de routes et supernetting	28
17 Décision de routage : le plus long préfixe	30
17.1 Lire une table route -n (capture de hta3p p. 12)	31
18 Couche réseau et modèle IPv4	32
18.1 Routage (p. 7 et 8)	32
19 En-tête IPv4	33
20 Fragmentation	35
20.1 Méthode de calcul (méthode générale)	35
21 Plans et signalisation	36
22 ICMP	37
22.1 Ping et traceroute (p. 6)	38
22.2 Sécurité (p. 7)	38
23 DHCP	39

23.1 Outils et démo (hta3p)	39
24 ARP	41
25 DNS	42
25.1 Résolution (p. 14 et 15)	42
25.2 Organisation, transport, sécurité (p. 16 à 18)	42
26 Outils de configuration réseau	43
26.1 Configuration permanente (p. 15 et 16)	43
27 Outils de diagnostic et de capture	45
28 Méthode d'examen	46
29 Formulaire	47

1. Télécommunication, partage du support, adressage

Source : hta1-intro.pdf, p. 4 à 25

La télécommunication, c'est partager de l'information à distance. Le cours retrace l'histoire (courrier, signaux optiques, télégraphe et code Morse, téléphone de Bell et Gray en 1876, standard de Tivadar Puskás en 1877, réseau téléphonique commuté PSTN, numéros E.164), puis passe au réseau informatique, sujet du cours. La téléphonie mobile est hors du cours (matière VIHAC04, p. 14).

Calcul de la slide p. 13 : débit de la téléphonie numérique (PCM)

Voix limitée à 4 kHz (bande téléphonique 0,3 à 3,4 kHz). Nyquist : échantillonner au double de la fréquence maximale, donc 8 kHz. Chaque échantillon quantifié sur 8 bits (256 niveaux, quantification logarithmique). Débit : $8000 \times 8 = 64\,000$ bit/s, soit 64 kbit/s.

Partager un support commun (p. 16 à 21)

Problème	Définition et variantes
Multiplexage	une même source combine plusieurs flux sur un canal (exemple : voie descendante d'une antenne mobile). FDM par fréquence (modulation sur porteuse), TDM par tranches de temps (impossible en analogique), CDM par codes orthogonaux sur la même bande et au même moment
Accès multiple	plusieurs sources indépendantes se partagent le canal (voie montante des mobiles). FDMA, TDMA (détection de porteuse ou synchronisation), CDMA (codes non parfaitement orthogonaux sans synchronisation)
Duplexage	émettre dans les deux sens : FDD (deux fréquences), TDD (chacun son tour), ou un fil par sens (Ethernet, USB)
Sauts multiples	des stations intermédiaires relaient le trafic

Arbitrage du canal (p. 21) : en multiplexage, l'émetteur attribue les canaux ; en accès multiple, l'allocation est fixe (fréquences en cellulaire), gérée par une entité centrale sur demande (tranches de temps en cellulaire), ou dis-

putée (Ethernet CSMA).

Modes de communication et adressage (p. 22 à 25)

- Point à point : unicast, anycast. Point à multipoint : broadcast, multicast (dont geocast).
- Adresse de destination pour désigner le récepteur, adresse source pour la réponse.
- Unicast : identifiant unique par nœud. Multicast : adresse de groupe, rejoindre et quitter le groupe. Geocast : coordonnées géographiques, les nœuds doivent se localiser.
- Attribution : manuelle, par un nœud responsable, ou choisie par chaque nœud. Avec des liens point à point, on adresse plutôt les interfaces, et une interface peut avoir plusieurs adresses.
- Modèle centralisé (efficace, mais à installer, à croire et vulnérable) ou distribué (consensus moins efficace, plus robuste). Exemples de la slide : cellulaire contre Wi-Fi, serveur HTTP contre BitTorrent, Skype avant et après 2011.

2. Protocoles, pile, modèles OSI et TCP/IP

Source : [hta1p-protocols.pdf](#), p. 2 à 8

Un protocole est un ensemble de règles (syntaxe, sémantique, algorithmes, automates, interfaces) qui permet à des entités d'échanger de l'information. On les empile en couches : chaque couche ne fait qu'une partie du travail et utilise la couche du dessous.

À retenir

- **SAP** (Service Access Point) : méthodes d'accès aux fonctions d'une couche, par exemple Connect(), Send(), Receive(). L'API réseau est typiquement les sockets BSD.
- **PDU** (Protocol Data Unit) : chaque couche dialogue « virtuellement » avec la même couche en face, mais ne parle réellement qu'aux couches voisines.
- **Encapsulation** : chaque couche ajoute son en-tête (parfois un pied) et ne touche qu'à ses propres en-têtes. Ordre sur le fil : en-tête 3, en-tête 2, en-tête 1, données.

OSI (ISO 7498)	PDU	TCP/IP	Rôle OSI
7 Application	Data		utilisateur de la pile
6 Présentation		Application	encodage, sérialisation, compression, chiffrement
5 Session			connexion, déconnexion, keepalive
4 Transport	Segment	Transport	échange fiable : segmenta-tion, acquittement, retrans-mission, multiplexage
3 Réseau	Packet	Network	entre deux nœuds quel-conques : relais, routage
2 Liaison	Frame	Link	vers un voisin (plusieurs voisins possibles)
1 Physique	Bit		bits sur le support

La slide p. 6 place la limite Application et Transport de TCP/IP au milieu

de la couche Session d'OSI. TCP/IP vient d'Unix BSD, définit des protocoles plutôt que des responsabilités de couche, et c'est lui qui est utilisé ; OSI est trop complexe et son implémentation n'existe que sur le papier. « Octet » plutôt que « byte » : le byte n'avait pas de taille fixe dans les années 1970.

Standardisation (p. 7 et 8)

Modèle boîte noire : on spécifie rigoureusement ce qui est visible de l'extérieur (interfaces, formats de PDU, séquences de messages), pas l'implémentation. Test de conformité : répondre aux entrées données par les sorties attendues ; entrées valides et invalides, couverture proche de 100 %, fuzzing.

Piège

Les p. 9 à 11 (IETF, IEEE, ITU, MPEG, ISO...) sont marquées « Not exam material ».

3. Sécurité et qualité de service

Source : htaip-protocols.pdf, p. 12 à 19

Sécurité (p. 12 à 15)

- Se conçoit dès le départ ; l'ajouter après coup est difficile.
- **Chiffrement** contre l'écoute. Échange de clés : Diffie et Hellman (1976), secret partagé sur un canal non sûr grâce à l'arithmétique modulaire.
- **Authentification** : identifier la source (signature) ; contre le rejeu, défi et réponse avec un aléa cryptographique. AAA : Authentication, Authorization, Access control.
- **Intégrité** : une simple somme de contrôle se recalcule ; il faut y inclure un secret partagé (hachage type SHA).

QoS (p. 16 à 19)

Réserver des ressources suppose une entité qui « possède » le réseau. IntServ (RFC 1633) réserve par flux, DiffServ (RFC 2475) par classe ; best effort, sans réservation, est le régime d'Internet. Limites négociées (Call Admission Control), imposées à l'entrée du réseau (UNI). RED (Random Early Detection) jette avec une certaine probabilité avant que le seau soit plein ; TCP ralentit alors.

	Token bucket	Leaky bucket
Mécanisme	+ K jetons/s, capacité B ; un message coûte M_i ; seau vide : rejet	un message ajoute M_i ; sortie $\leq K$ jetons/s ; seau plein : rejet
Effet	mesure, passe ou échoue ; le débit peut dépasser K brièvement	mise en forme : sortie $\leq K$, il faut une file
Dimensionnement	K = débit moyen, B = (max – moyen) $\times$ durée de rafale	

Exemple chiffré (valeurs choisies) : moyen 2 Mbit/s, max 10 Mbit/s, rafales de 0,5 s : $K = 2$ Mbit/s, $B = (10 - 2) \times 0,5 = 4$ Mbit.

4. Multiplexage statistique

Source : htaip-protocols.pdf, p. 20

Question de la slide : $N = 500$ usagers, chacun actif avec probabilité $p = 1\%$. Combien de liens L faut-il ?

À retenir

- Moyenne : $Np = 5$ usagers actifs.
- Nombre d'actifs X : loi binomiale (N essais, succès p), approchée par une loi de Poisson de paramètre $\lambda = Np$ quand N est grand et p petit.
- Probabilité de blocage avec L liens : $\Pr(X > L) = 1 - \text{CDF}(X, L)$.
- Inégalité de Bienaymé et Tchebychev : $\Pr(|X - \mu| \geq k\sigma) \leq 1/k^2$, vraie pour toute loi, donc elle surestime.
- Modèle plus réaliste pour la téléphonie : Erlang (probabilité d'appel, durée moyenne).

Piège

La slide écrit $\sigma = Np(1 - p)$ pour la binomiale et $\sigma = \lambda$ pour Poisson. Ce sont les *variances*. Les écarts types sont $\sigma = \sqrt{Np(1 - p)}$ et $\sigma = \sqrt{\lambda}$. La slide en déduit $L = \lambda + 3\lambda = 20$ et $\Pr(|X - 5| \geq 15) \leq 1/9$. Avec $\sigma = \sqrt{5} \approx 2,24$, on aurait $L = 5 + 3\sqrt{5} \approx 11,7$ et, pour $L = 20$, $k = 15/\sqrt{5} \approx 6,7$, soit une borne $1/k^2 = 1/45 \approx 0,022$. Quelle version l'enseignant attend à l'examen : **À VÉRIFIER**.

Valeurs exactes (calculées en Python, pour se situer)

Poisson, $\lambda = 5$: $\Pr(X > 20) \approx 8,1 \times 10^{-8}$; binomiale exacte : $\approx 6,3 \times 10^{-8}$. Avec 11 liens, Poisson donne $\Pr(X > 11) \approx 0,0055$. Tchebychev donne une borne très au-dessus de ces valeurs : c'est le prix de sa généralité.

5. Sérialisation : TLV, XML, JSON, YAML

Source : `hta1p-protocols.pdf`, p. 21 à 26

Couche présentation. Les machines ne représentent pas les données de la même façon : entiers 32 ou 64 bits, ordre des octets (Intel en little endian, le réseau en big endian, conversions `htons()`, `htonl()`, `ntohs()`, `ntohl()`), flottants. Il faut une représentation indépendante de la machine. Critères : lisible par un humain, auto-descriptive, extensible (sauter ce qu'on ne connaît pas), résistante aux erreurs, lisible à partir du milieu.

À retenir

- **TLV** (Type, Length, Value) : la longueur connue permet de sauter les éléments inconnus, donc une nouvelle version peut ajouter des types. Type composé : la valeur est une suite de TLV. DHCP code ses options en TLV (`hta3-ipv4-protocols.pdf`, p. 8).
- **ASN.1** (ITU-T, X.680) : langage de description d'interfaces, sérialisé en TLV. Règles BER, CER, DER (X.690, faciles à analyser), PER (X.691, compact). Utilisé par SS7, SNMP, la signalisation 2G à 5G (PER), les certificats X.509 (DER).
- **XML** : langage à balises issu de HTML ; validation par DTD ou XSD, XPath, XSLT, espaces de noms, API SAX. L'encodage XER d'ASN.1 est du XML (X.693).
- **JSON** : scalaires, tableaux `[]`, objets `{ }` ; `JSON.parse()` plutôt que `eval()`. L'encodage JER d'ASN.1 est du JSON (X.697).
- **YAML** : hiérarchie par indentation, listes par tiret, dictionnaires clé : valeur. Sur-ensemble de JSON. Défauts : pas de validation de schéma, erreurs d'indentation, troncature indétectable (marqueurs de début et de fin optionnels). Surtout pour les fichiers de configuration.

Piège

L'exemple ASN.1 détaillé de la p. 23 (spécification et encodage DER octet par octet) est marqué « Not exam material ». La conversion XML vers JSON ou YAML, elle, est annoncée comme question possible (p. 26).

Méthode : convertir du XML en JSON puis en YAML (méthode générale)

1. L'élément racine englobe tout : son contenu devient l'objet JSON de premier niveau (la slide p. 25 omet le nom de la racine NTALecture).
2. Un élément qui ne contient que du texte devient une paire clé et valeur : "seq": 2. Un nombre peut s'écrire sans guillemets, le texte entre guillemets.
3. Un élément qui contient des éléments différents devient un objet { }.
4. Des éléments frères de même nom deviennent un tableau [] ; la balise répétée disparaît (la slide transforme les SlideTitle en simple liste).
5. YAML : mêmes données, on retire accolades, crochets et guillemets inutiles ; un niveau d'indentation par niveau d'imbrication ; chaque élément de liste commence par un tiret.
6. Les attributs XML n'ont pas d'équivalent imposé : choisir une convention et l'écrire (question non traitée par les slides).

Exemple de la slide (p. 24 à 26)

```
<NTALecture>
  <seq>2</seq>
  <title>Protocols Practical</title>
  <slides>
    <SlideTitle>Title</SlideTitle>
    <SlideTitle>Practical</SlideTitle>
    <SlideTitle>Buckets</SlideTitle>
    <SlideTitle>Multiplexing</SlideTitle>
  </slides>
</NTALecture>
```

JSON :

```
{
  "seq": 2,
  "title": "Protocols Practical",
  "slides": [ "Title",
    "Practical", "Buckets",
    "Multiplexing" ]
}
```

YAML :

```
seq: 2
title: Protocols Practical
slides:
  - Title
  - Practical
  - Buckets
  - Multiplexing
```

6. Binaire, décimal, hexadécimal

Source : méthode générale; utilisée dans hta2p-ipv4-practical.pdf, p. 6 et 11 à 14

Une adresse IPv4 est une suite de 32 bits. Toutes les méthodes de calcul reposent sur trois bases : binaire (base 2), décimal (base 10), hexadécimal (base 16, chiffres 0 à 9 puis A à F).

À retenir

Puissances de 2 d'un octet, à savoir par cœur, de gauche à droite :

Rang du bit	7	6	5	4	3	2	1	0
Poids	128	64	32	16	8	4	2	1

Un octet vaut de 0 (00000000) à 255 (11111111). $2^{10} = 1024$, $2^{16} = 65\,536$, $2^{24} = 16\,777\,216$, $2^{32} = 4\,294\,967\,296$.

Décimal vers binaire, méthode des poids (soustractions)

Parcourir les poids de 128 à 1 : si le reste est supérieur ou égal au poids, écrire 1 et soustraire ; sinon écrire 0.

172 en binaire

Poids	128	64	32	16	8	4	2	1
Reste avant	172	44	44	12	12	4	0	0
Bit	1	0	1	0	1	1	0	0
Reste après	44	44	12	12	4	0	0	0

$172 = 128 + 32 + 8 + 4 = 10101100$. C'est le premier octet de 172.16.0.0, écrit 10101100 sur la slide p. 8 de hta2p.

Décimal vers binaire, divisions successives par 2

Diviser par 2 jusqu'à obtenir 0 ; les restes lus *de bas en haut* donnent le nombre binaire.

77 en binaire

Dividende		Quotient	Reste
77	÷2	38	1
38	÷2	19	0
19	÷2	9	1
9	÷2	4	1
4	÷2	2	0
2	÷2	1	0
1	÷2	0	1

Lecture de bas en haut : 1001101, complété à 8 bits : 01001101. Contrôle par les poids : $64 + 8 + 4 + 1 = 77$.

Binaire vers décimal

Additionner les poids des bits à 1.

10110110 et 11100000

$10110110 = 128 + 32 + 16 + 4 + 2 = 182$. $11100000 = 128 + 64 + 32 = 224$.

Piège

La slide hta2p p. 11 écrit « 224 is 1110000 » : il manque un zéro. Un octet a toujours 8 bits, 224 s'écrit 11100000. Toujours compléter à 8 bits par des zéros à gauche.

Hexadécimal

Un chiffre hexadécimal code exactement 4 bits (un quartet), donc un octet s'écrit avec deux chiffres hexadécimaux.

Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Déc.	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Bin.	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

(Lire le binaire de haut en bas : B = 1011.)

- **Binaire vers hex** : couper en quartets à partir de la droite, traduire chacun. 1010 1100 → A, C : 172 = 0xAC.
- **Hex vers binaire** : chaque chiffre donne 4 bits. 0x45 → 0100 0101.
- **Hex vers décimal** : $\text{chiffre}_1 \times 16 + \text{chiffre}_0$. 0xC8 = $12 \times 16 + 8 = 200$. Sur 16 bits : 0x6F89 = $6 \times 4096 + 15 \times 256 + 8 \times 16 + 9 = 28\,553$ (l'Identification de la capture Wireshark, hta3p p. 22).
- **Décimal vers hex** : divisions successives par 16, restes lus de bas en haut.

3000 en hexadécimal

$3000 = 187 \times 16 + 8$; $187 = 11 \times 16 + 11$; $11 = 0 \times 16 + 11$. Restes de bas en haut : 11, 11, 8, soit B, B, 8 : 3000 = 0xBB8. Contrôle : $11 \times 256 + 11 \times 16 + 8 = 2816 + 176 + 8 = 3000$.

7. Adresse IPv4 et notation pointée

Source : hta2-ipv4.pdf, p. 6 ; hta2p-ipv4-practical.pdf, p. 2

À retenir

- 32 bits, soit 4 octets. Notation décimale pointée : chaque octet écrit en décimal (0 à 255), séparés par des points. Exemple de la slide : 152.66.244.17.
- Deux parties : **préfixe réseau** (à gauche) et **identifiant d'hôte** (à droite). Où passe la frontière : voir classes (concept 8) puis CIDR (concept 10).
- L'adresse identifie une *interface*, pas un nœud ; une interface peut avoir plusieurs adresses.
- Les nœuds de même préfixe que moi sont joignables directement ; les autres, via la passerelle configurée.
- Routage en deux phases : atteindre le réseau cible grâce au préfixe, puis trouver l'hôte dans ce réseau.
- Modes : unicast (usage normal), multicast (groupe défini avec IGMP), broadcast (seulement dans le sous-réseau).

152.66.244.17 en binaire et en hexadécimal

Octet	152	66	244	17
Binaire	10011000	01000010	11110100	00010001
Hex	98	42	F4	11

152 = 128 + 16 + 8 ; 66 = 64 + 2 ; 244 = 128 + 64 + 32 + 16 + 4 ; 17 = 16 + 1.
En hexadécimal sur 32 bits : 0x9842F411.

Piège

Un octet ne dépasse jamais 255. « 152.66.140.266 » n'est pas une adresse : la slide hta2p p. 12 le classe « invalid ».

8. Classes historiques

Source : hta2p-ipv4-practical.pdf, p. 3 à 5

Adressage d’origine (années 1980) : la position de la frontière se lit dans les premiers bits.

Classe	Premiers bits	Préfixe	1 ^{er} octet	Usage
A	0	8 bits	1 à 126	unicast
B	10	16 bits	128 à 191	unicast
C	110	24 bits	192 à 223	unicast
D	1110		224 à 239	multicast
E	11110		240 et plus	réservé

Plages A, B, C : slide p. 4. D et E se déduisent des premiers bits (11100000 = 224, 11101111 = 239) ; la slide ne donne pas la plage exacte de E.

Classe	Réseaux	Hôtes par réseau	Justification
A	126	16 777 214	$2^7 = 128$ moins 0 et 127 ; $2^{24} - 2$
B	16 384	65 534	2^{14} (16 bits moins 2 fixés) ; $2^{16} - 2$
C	2 097 152	254	2^{21} (24 bits moins 3 fixés) ; $2^8 - 2$

À retenir

- Le réseau 127 est réservé au loopback.
- Partie hôte tout à 0 : « ce réseau entier » (adresse de réseau). Tout à 1 : broadcast. D’où le « -2 » dans chaque nombre d’hôtes.

Pourquoi les classes ont été abandonnées (p. 5)

Classe A trop grande : on la découpe par service (**subnetting**, trois niveaux : réseau, sous-réseau, hôte). Classe C trop petite : 2000 machines demanderaient 8 blocs C. Les petits blocs ajoutés multiplient les préfixes et les tables de routage explosent. Réponse : CIDR.

9. Adresses spéciales et privées

Source : hta2p-ipv4-practical.pdf, p. 4 et 8

Plage	Rôle
127.0.0.1	loopback (l'interface locale elle-même)
10.0.0.0/8	privée : un réseau de classe A
172.16.0.0/12	privée : 16 réseaux de classe B contigus (172.16 à 172.31), masque 255.240.0.0
192.168.0.0/16	privée : 256 réseaux de classe C contigus
hôte tout à 0	adresse du réseau
hôte tout à 1	broadcast du réseau

Les adresses privées ne sont pas routables sur Internet (on passe par du NAT) et on les découpe couramment en sous-réseaux plus petits.

Pourquoi 172.16.0.0/12 couvre 172.16 à 172.31

Slide p. 8 : 10101100.0001xxxx.y.z. Les 12 premiers bits sont fixés ; les 4 bits xxxx du deuxième octet varient de 0000 à 1111, donc le deuxième octet va de 00010000 = 16 à 00011111 = 31. 172.32.0.1 est hors de la plage : publique.

Piège

La slide écrit « 10.0.0.0/24 » pour « one Class A network » et « 127.0.0.0/24 » pour le loopback. Un réseau de classe A a un préfixe de 8 bits, et les RFC 1918 (privées) et 1122 (loopback) donnent 10.0.0.0/8 et 127.0.0.0/8. Coquille probable de la slide ; ce document écrit /8. Confirmation auprès de l'enseignant : **À VÉRIFIER.**

Deux adresses vues dans les slides, sans y être expliquées. 0.0.0.0 avec le masque 0.0.0.0 est la *route par défaut* dans la table route -n (hta3p p. 12). Le DHCP Discover part en « IP broadcast » (hta3 p. 10) : l'adresse utilisée est 255.255.255.255, broadcast limité (RFC 2131, hors slides).

10. Masque de sous-réseau et notation CIDR

Source : hta2p-ipv4-practical.pdf, p. 6, 7 et 11

CIDR (Classless Inter-Domain Routing, 1992) : la frontière réseau et hôte peut être n'importe où. Deux notations équivalentes : a.b.c.d/n (longueur de préfixe n) ou un masque, n bits à 1 suivis de $32 - n$ bits à 0. Exemple de la slide : $n = 14$ donne 255.252.0.0, car $252 = 11111100$.

Masque /14 255.252.0.0 11111111.111111|00.00000000.00000000

À retenir

Valeurs possibles d'un octet de masque (nombre de 1 en tête) :

Bits à 1	0	1	2	3	4	5	6	7	8
Octet	0	128	192	224	240	248	252	254	255

Chaque valeur ajoute le poids suivant : 128, +64 = 192, +32 = 224, +16 = 240...

/n vers masque

Écrire $n = 8 + 8 + \dots + r$ avec $r < 8$: autant d'octets 255 que de 8, puis l'octet qui a r bits à 1, puis des 0.

/19 et /13 (le premier est l'exemple 1 de la slide p. 11)

19 = 8 + 8 + 3 : 255.255, puis 3 bits à 1 = 224, puis 0 : **255.255.224.0**.

13 = 8 + 5 : 255, puis 5 bits à 1 = 248 : **255.248.0.0**.

Masque vers /n

Compter les 1 : 8 par octet 255, plus les bits de l'octet partiel (table ci-dessus).

255.224.0.0 (slide p. 11) et 255.255.255.240

255.224.0.0 : 8 + 3 (224 = 11100000) = **/11**.

255.255.255.240 : 8 + 8 + 8 + 4 = **/28**.

Piège

Un masque doit être une suite de 1 *puis* une suite de 0. 255.96.0.0 est invalide (slide p. 11) : 96 = 01100000, un 0 précède des 1. De même 255.255.255.160 (160 = 10100000) ou 255.0.255.0.

Taille d'un réseau

Un préfixe / n laisse $h = 32 - n$ bits d'hôte : 2^h adresses, dont $2^h - 2$ attribuables (on retire réseau et broadcast). C'est le tableau de la slide p. 7, prolongé ci-dessous.

/n	Masque	Bloc	Hôtes	/n	Masque	Bloc	Hôtes
/8	255.0.0.0	2^{24}	16 777 214	/24	255.255.255.0	256	254
/16	255.255.0.0	2^{16}	65 534	/25	255.255.255.128	128	126
/17	255.255.128.0	2^{15}	32 766	/26	255.255.255.192	64	62
/18	255.255.192.0	2^{14}	16 382	/27	255.255.255.224	32	30
/19	255.255.224.0	2^{13}	8 190	/28	255.255.255.240	16	14
/20	255.255.240.0	2^{12}	4 094	/29	255.255.255.248	8	6
/21	255.255.248.0	2^{11}	2 046	/30	255.255.255.252	4	2
/22	255.255.252.0	1 024	1 022				
/23	255.255.254.0	512	510				

Plus petit préfixe pour H hôtes (méthode générale) : chercher le plus petit h tel que $2^h - 2 \geq H$, puis $n = 32 - h$. Pour 300 hôtes : $2^8 - 2 = 254$ ne suffit pas, $2^9 - 2 = 510$ suffit, donc /23. Pour 62 hôtes : /26 ($62 \geq 62$). Pour 63 : /25.

Piège

Les préfixes /31 et /32 ne suivent pas la règle « -2 » ; les slides n'en parlent pas. La table route -n de hta3p p. 12 montre des routes vers un seul hôte, masque 255.255.255.255 (drapeau H).

11. Réseau, broadcast, plage d'hôtes : méthode du ET

Source : méthode générale ; alignement binaire comme sur [hta2p-ipv4-practical.pdf](#), p. 12 à 14

À retenir

1. Écrire l'adresse et le masque en binaire, alignés.
2. **Adresse réseau** = adresse ET masque, bit à bit : on garde le préfixe, on met la partie hôte à 0.
3. **Broadcast** = préfixe inchangé, partie hôte tout à 1.
4. **Première adresse d'hôte** = réseau +1 ; **dernière** = broadcast -1.
5. **Nombre d'hôtes** = $2^{32-n} - 2$.

ET logique : $1 \wedge 1 = 1$, sinon 0. Seul l'octet où le masque n'est ni 255 ni 0 demande un calcul ; ailleurs on recopie (masque 255) ou on met 0 (masque 0).

Exemple 1 : 192.168.10.77/26

Adresse	192.168.10.77	11000000.10101000.00001010.01 001101
Masque /26	255.255.255.192	11111111.11111111.11111111.11 000000
Réseau (ET)	192.168.10.64	11000000.10101000.00001010.01 000000
Broadcast	192.168.10.127	11000000.10101000.00001010.01 111111

Octet intéressant : le 4^e. 77 = 01001101, masque 192 = 11000000. ET : 01000000 = 64. Broadcast : 01111111 = 127.

Réseau **192.168.10.64**, broadcast **192.168.10.127**, hôtes de 192.168.10.65 à 192.168.10.126, soit $2^6 - 2 = 62$ hôtes.

Exemple 2 : 152.66.163.9/18 (adresse de la slide p. 12)

Adresse	152.66.163.9	10011000.01000010.10 100011.00001001
Masque /18	255.255.192.0	11111111.11111111.11 000000.00000000
Réseau (ET)	152.66.128.0	10011000.01000010.10 000000.00000000
Broadcast	152.66.191.255	10011000.01000010.10 111111.11111111

Masque 255.255.192.0. Octet intéressant : le 3^e. 163 = 10100011, 192 = 11000000, ET : 10000000 = 128. Broadcast : 10111111.11111111, soit

191.255.

Réseau **152.66.128.0**, broadcast **152.66.191.255**, hôtes de 152.66.128.1 à 152.66.191.254, soit $2^{14} - 2 = 16\,382$ hôtes.

Exemple 3 : 10.45.200.13/12

Adresse	10.45.200.13	00001010.0010 1101.11001000.00001101
Masque /12	255.240.0.0	11111111.1111 0000.00000000.00000000
Réseau (ET)	10.32.0.0	00001010.0010 0000.00000000.00000000
Broadcast	10.47.255.255	00001010.0010 1111.11111111.11111111

Masque 255.240.0.0. Octet intéressant : le 2^e. 45 = 00101101, 240 = 11110000, ET : 00100000 = 32. Les octets 3 et 4 passent à 0 (réseau) ou 255 (broadcast). Broadcast du 2^e octet : 00101111 = 47.

Réseau **10.32.0.0**, broadcast **10.47.255.255**, hôtes de 10.32.0.1 à 10.47.255.254, soit $2^{20} - 2 = 1\,048\,574$ hôtes.

Exemple 4 : 172.20.37.200/21

Adresse	172.20.37.200	10101100.00010100.00100 101.11001000
Masque /21	255.255.248.0	11111111.11111111.1111 000.00000000
Réseau (ET)	172.20.32.0	10101100.00010100.00100 000.00000000
Broadcast	172.20.39.255	10101100.00010100.00100 111.11111111

Masque 255.255.248.0. 37 = 00100101, 248 = 11111000, ET : 00100000 = 32. Broadcast : 00100111 = 39, puis 255.

Réseau **172.20.32.0**, broadcast **172.20.39.255**, hôtes de 172.20.32.1 à 172.20.39.254, soit $2^{11} - 2 = 2046$ hôtes.

Exemple 5 : 200.1.1.130/30, lien point à point

Adresse	200.1.1.130	11001000.00000001.00000001.100000 10
Masque /30	255.255.255.252	11111111.11111111.11111111.111111 00
Réseau (ET)	200.1.1.128	11001000.00000001.00000001.100000 00
Broadcast	200.1.1.131	11001000.00000001.00000001.100000 11

130 = 10000010, 252 = 11111100, ET : 10000000 = 128. Réseau **200.1.1.128**, broadcast **200.1.1.131**, deux hôtes : 200.1.1.129 et 200.1.1.130.

12. Méthode rapide du nombre magique

Source : méthode générale, équivalente à la méthode du ET

Pour calculer vite sans écrire 32 bits.

À retenir

1. Repérer l'**octet intéressant** : celui où le masque n'est ni 255 ni 0.
2. **Nombre magique** $m = 256 - \text{octet du masque (taille du bloc)}$.
3. Les réseaux commencent aux multiples du nombre magique : 0, m , $2m$...
4. Réseau : le plus grand multiple de m inférieur ou égal à la valeur de l'octet ; octets à droite à 0.
5. Broadcast : réseau $+m - 1$ dans l'octet intéressant ; octets à droite à 255.

Les exemples de la page précédente, en une ligne chacun

192.168.10.77/26	4 ^e octet, $m = 256 - 192 = 64$. Multiples : 0, 64, 128. $64 \leq 77 < 128$: réseau .64, broadcast $64 + 63 = 127$.
152.66.163.9/18	3 ^e octet, $m = 256 - 192 = 64$. $128 \leq 163 < 192$: réseau 152.66.128.0, broadcast 152.66.191.255.
10.45.200.13/12	2 ^e octet, $m = 256 - 240 = 16$. Multiples 32, $48 : 32 \leq 45 < 48$: réseau 10.32.0.0, broadcast 10.47.255.255.
172.20.37.200/21	3 ^e octet, $m = 256 - 248 = 8$. $32 \leq 37 < 40$: réseau 172.20.32.0, broadcast 172.20.39.255.

Piège

- Préfixe multiple de 8 (/8, /16, /24) : pas d'octet intéressant, octets hôte à 0.
- Un octet à droite de l'octet intéressant vaut 0 pour le réseau et 255 pour le broadcast, pas « $m - 1$ ».
- Une adresse de réseau ou de broadcast ne s'attribue pas : 193.225.14.63/28 est le broadcast de 193.225.14.48/28 ($48 + 15 = 63$).

13. Appartenance d'une adresse à un réseau

Source : hta2p-ipv4-practical.pdf, p. 12 (exemple 2)

À retenir

Deux adresses sont dans le même sous-réseau si et seulement si leurs n premiers bits sont identiques, autrement dit si (adresse 1 ET masque) = (adresse 2 ET masque). Méthode rapide : calculer la plage du réseau de référence et vérifier que l'adresse y tombe.

Exemple 2 de la slide : lesquelles sont sur le même sous-réseau que 152.66.163.9/18?

Nous	152.66.163.9	10011000.01000010.10 100011.00001001
(1)	152.66.178.180	10011000.01000010.10 110010.10110100
(2)	162.66.131.11	10100010.01000010.10 000011.00001011
(3)	152.66.195.210	10011000.01000010.11 000011.11010010
(4)	152.66.129.51	10011000.01000010.10 000001.00110011
(5)	152.66.140.266	10011000.01000010.10 001100.XXXXXXXXXX

152.66 est commun à toutes sauf (2) ; on compare les 2 premiers bits du 3^e octet, 10 pour la référence.

- (1) 152.66.178.180 : 178 = 10110010, commence par 10 : **oui**.
- (2) 162.66.131.11 : le premier octet diffère (162 et non 152) : **non**.
- (3) 152.66.195.210 : 195 = 11000011, commence par 11 : **non** (réseau 152.66.192.0/18).
- (4) 152.66.129.51 : 129 = 10000001 : **oui**.
- (5) 152.66.140.266 : 266 dépasse 255, adresse **invalid**.

Réponse de la slide : 1 et 4. Méthode rapide : plage 152.66.128.0 à 152.66.191.255, 178 et 129 y sont, 195 non.

Piège

192.168.5.130/25 et 192.168.5.100/25 sont dans deux réseaux (.128/25 et .0/25) : ils passent par la passerelle. En /24, ils se joindraient directement.

14. Découpage en sous-réseaux de taille fixe

Source : principe : hta2p-ipv4-practical.pdf, p. 5 et 8 ; méthode générale

Subnetting : on emprunte les premiers bits de la partie hôte pour numéroté des sous-réseaux.

À retenir

- Emprunter s bits donne 2^s sous-réseaux de préfixe $n + s$.
- Pour S sous-réseaux : plus petit s tel que $2^s \geq S$.
- Pour au moins H hôtes par sous-réseau : plus petit h tel que $2^h - 2 \geq H$, puis préfixe $32 - h$.
- Sous-réseau k (compté depuis 0) : base + $k \times 2^{32-(n+s)}$.

192.168.20.0/24 en au moins 6 sous-réseaux égaux

$4 < 6 \leq 8 = 2^3$: $s = 3$ bits, /27, 255.255.255.224, pas de 32, 30 hôtes chacun.

Réseau /24	192.168.20.0	11000000.10101000.00010100.	00000000
Masque /27	255.255.255.224	11111111.11111111.11111111.	111 00000

Sous-rés. 0	192.168.20.0	11000000.10101000.00010100.000	00000
Sous-rés. 1	192.168.20.32	11000000.10101000.00010100.001	00000
Sous-rés. 2	192.168.20.64	11000000.10101000.00010100.010	00000

k	Réseau /27	Bcast	k	Réseau /27	Bcast
0	192.168.20.0	.31	4	192.168.20.128	.159
1	192.168.20.32	.63	5	192.168.20.160	.191
2	192.168.20.64	.95	6	192.168.20.192	.223
3	192.168.20.96	.127	7	192.168.20.224	.255

Hôtes : de réseau +1 à broadcast -1 (.33 à .62 pour $k = 1$). Deux restent libres.

172.16.0.0/16 en sous-réseaux d'au moins 500 hôtes

$254 < 500 \leq 510 = 2^9 - 2$: préfixe /23 (255.255.254.0), 7 bits emprun-

tés, $2^7 = 128$ sous-réseaux de 512 adresses, pas de 2 dans le 3^e octet : 172.16.0.0, 172.16.2.0...172.16.254.0/23.

Sous-réseau $k = 36 : 36 \times 2 = 72$, **172.16.72.0/23**, hôtes 172.16.72.1 à 172.16.73.254, broadcast 172.16.73.255.

Base /16	172.16.0.0	10101100.00010000.	00000000.00000000
----------	------------	--------------------	-------------------

Masque /23	255.255.254.0	11111111.11111111.11111111	0.00000000
------------	---------------	----------------------------	------------

$k = 36$	172.16.72.0	10101100.00010000.0100100	0.00000000
----------	-------------	---------------------------	------------

Broadcast	172.16.73.255	10101100.00010000.0100100	1.11111111
-----------	---------------	---------------------------	------------

Vérification en binaire : les 7 bits empruntés (bits 17 à 23) valent 0100100, soit $32 + 4 = 36$; le dernier bit du 3^e octet et tout le 4^e octet forment la partie hôte (9 bits), d'où 72.0 pour le réseau et 73.255 pour le broadcast.

15. VLSM : sous-réseaux de tailles différentes

Source : méthode générale ; les slides disent seulement qu'avec CIDR « we can have a network of any size » (hta2p, p. 6)

VLSM (Variable Length Subnet Masking) : chaque sous-réseau reçoit le préfixe adapté à son besoin, pour ne pas gaspiller d'adresses.

À retenir

1. Classer les besoins du plus grand au plus petit.
2. Pour chacun, prendre le plus petit bloc 2^h avec $2^h - 2 \geq$ besoin.
3. Allouer à la suite, en partant de l'adresse de base. En commençant par les plus gros blocs, chaque bloc tombe automatiquement sur un multiple de sa taille (alignement).
4. Vérifier que le total ne dépasse pas le bloc de départ et qu'aucun bloc ne chevauche un autre.

192.168.50.0/24 : LAN de 100, 50, 20 et 10 hôtes, deux liens point à point

Tailles : 100 $\rightarrow$ 128 (/25), 50 $\rightarrow$ 64 (/26), 20 $\rightarrow$ 32 (/27), 10 $\rightarrow$ 16 (/28), 2 $\rightarrow$ 4 (/30) deux fois. Total $128 + 64 + 32 + 16 + 4 + 4 = 248 \leq 256$: ça tient.

Besoin	Réseau	Hôtes utilisables	Broadcast	Hôtes
100	192.168.50.0/25	.1 à .126	.127	126
50	192.168.50.128/26	.129 à .190	.191	62
20	192.168.50.192/27	.193 à .222	.223	30
10	192.168.50.224/28	.225 à .238	.239	14
2	192.168.50.240/30	.241 et .242	.243	2
2	192.168.50.244/30	.245 et .246	.247	2

Reste libre : 192.168.50.248/29 (de .248 à .255).

Piège

Allouer dans le désordre casse l'alignement : un /25 ne commence qu'à .0 ou .128 ; un /30 placé d'abord en .0 repousse le /25 à .128. Si la somme des blocs dépasse l'espace, le plan est impossible : le dire.

16. Agrégation de routes et supernetting

Source : [hta2p-ipv4-practical.pdf](#), p. 6 et 14 (exemple 4)

Moins de lignes dans la table de routage, c'est une recherche plus rapide (slide p. 14).

À retenir

Deux préfixes de même longueur n se fusionnent en un préfixe $n - 1$ si :

- seul le dernier bit de leurs préfixes diffère (ils sont adjacents *et* alignés) ;
- ils ont le même saut suivant (même interface).

On répète tant que c'est possible. Exemple de la slide p. 6 : deux /18 adjacents donnent un /17 « si les 17 premiers bits sont identiques et le saut suivant aussi ».

Exemple 4 de la slide : quatre routes vers eth5

```
(1) /10 130.120.0.0 10000010.01|111000.00000000.00000000
(2) /10 130.128.0.0 10000010.10|000000.00000000.00000000
(3) /11 130.192.0.0 10000010.110|000000.00000000.00000000
(4) /11 130.224.0.0 10000010.111|000000.00000000.00000000
```

- (3) et (4) : préfixes de 11 bits, seul le 11^e bit diffère (0 et 1). Plages 130.192.0.0 à 130.223.255.255 et 130.224.0.0 à 130.255.255.255, contiguës : fusion en **130.192.0.0/10**.
- (2) 130.128.0.0/10 et ce 130.192.0.0/10 : seul le 10^e bit diffère : fusion en **130.128.0.0/9**.
- (1) « 130.120.0.0/10 » est mal écrit : 120 = 01111000, les 2 premiers bits du 2^e octet valent 01, donc le vrai réseau est 130.64.0.0/10 (de 130.64.0.0 à 130.127.255.255). Avec 130.128.0.0/10, les deux derniers bits du préfixe diffèrent (01 et 10) : pas de fusion.

Résultat de la slide : (2), (3) et (4) deviennent 130.128.0.0/9 ; la table passe de 4 à 2 lignes.

Quatre /24 consécutifs

/24	192.168.4.0	11000000.10101000.000001 00.00000000
/24	192.168.5.0	11000000.10101000.000001 01.00000000
/24	192.168.6.0	11000000.10101000.000001 10.00000000
/24	192.168.7.0	11000000.10101000.000001 11.00000000

Résumé /22 192.168.4.0 11000000.10101000.000001|00.00000000

Les 22 premiers bits sont communs, les 2 bits suivants prennent les 4 valeurs : le résumé exact est **192.168.4.0/22**.

Deux /24 adjacents mais non alignés

/24 192.168.1.0 11000000.10101000.000000|01.00000000

/24 192.168.2.0 11000000.10101000.000000|10.00000000

192.168.1.0/24 et 192.168.2.0/24 sont contigus, mais leurs 3^{es} octets 00000001 et 00000010 diffèrent sur les deux derniers bits : pas de /23 commun. Le plus petit préfixe qui les couvre, 192.168.0.0/22, contient aussi 192.168.0.0/24 et 192.168.3.0/24 : ce n'est plus une agrégation exacte.

17. Décision de routage : le plus long préfixe

Source : hta2p-ipv4-practical.pdf, p. 13 (exemple 3) ; hta3p-ipv4-tools.pdf, p. 12

À retenir

1. Pour chaque ligne de la table : la destination appartient-elle au préfixe ? (ET avec le masque de la ligne, comparer.)
2. Parmi les lignes qui correspondent, garder celle de **préfixe le plus long** (longest prefix match) : c'est la plus spécifique.
3. La route par défaut 0.0.0.0/0 correspond à tout, avec la longueur 0 : elle ne sert que si rien d'autre ne correspond.

Exemple 3 de la slide : destination 170.85.167.2

Dest.	170.85.167.2	10101010.010101 01.10100111.00000010
/13 eth7	170.80.0.0	10101010.01010 000.00000000.00000000
/13 eth2	170.88.0.0	10101010.01011 000.00000000.00000000
/14 eth11	170.84.0.0	10101010.010101 00.00000000.00000000

85 = 01010101. Comparer les bits du 2^e octet sur la longueur de chaque préfixe :

- 170.80.0.0/13 (eth7) : 5 bits du 2^e octet, 01010 contre 01010 : correspond.
- 170.88.0.0/13 (eth2) : 01011 contre 01010 : non.
- 170.84.0.0/14 (eth11) : 6 bits, 010101 contre 010101 : correspond.

Lignes 1 et 3 correspondent ; /14 est plus long que /13 : sortie **eth11**.

Table avec route par défaut (exemple construit pour l'entraînement)

Table : 0.0.0.0/0 eth0 ; 10.0.0.0/8 eth1 ; 10.1.0.0/16 eth2 ; 10.1.4.0/22 eth3 ; 10.1.5.128/25 eth4.

Dest.	10.1.5.20	00001010.00000001.00000101.0 0010100
/8 eth1	10.0.0.0	00001010. 00000000.00000000.00000000
/16 eth2	10.1.0.0	00001010.00000001. 00000000.00000000
/22 eth3	10.1.4.0	00001010.00000001.000001 00.00000000
/25 eth4	10.1.5.128	00001010.00000001.00000101.1 00000000

- 10.1.5.20 : /8, /16 et /22 (plage 10.1.4.0 à 10.1.7.255) correspondent ; /25 couvre .128 à .255 du 10.1.5, et 20 n'y est pas. Sortie **eth3**.
- 10.1.5.200 : le /25 correspond aussi : **eth4**.
- 10.1.8.1 : le /22 couvre 10.1.4.x à 10.1.7.x, 8 en sort : **eth2**.
- 10.2.0.1 : seul /8 : **eth1**. 8.8.8.8 : route par défaut, **eth0**.

Lire une table route -n (capture de hta3p p. 12)

Destination	Gateway	Genmask	Flags	Iface
0.0.0.0	192.168.100.254	0.0.0.0	UG	enp8s0f0
8.8.8.8	192.168.100.254	255.255.255.255	UGH	enp8s0f0
10.1.33.0	0.0.0.0	255.255.255.0	U	enp3s0
172.17.0.0	0.0.0.0	255.255.0.0	U	docker0
192.168.100.0	0.0.0.0	255.255.255.0	U	enp8s0f0

(extrait ; colonnes Metric, Ref, Use omises). Gateway 0.0.0.0 : réseau directement connecté. Drapeaux, selon la page de manuel de route (non expliqués sur la slide) : U route active, G passe par une passerelle, H destination d'un seul hôte. Un paquet vers 10.1.33.2 sort par enp3s0 sans passerelle ; vers 9.9.9.9, seule la route par défaut correspond : il part vers 192.168.100.254.

18. Couche réseau et modèle IPv4

Source : hta2-ipv4.pdf, p. 2 à 9 et 17

À retenir

- Couche réseau : échange de paquets entre deux nœuds quelconques. Adressage (identifiant unique), routage (trouver un chemin), relaiage : {DST, SRC, saut précédent} → saut suivant.
- Commutation de circuits : circuit virtuel à établir avant d'envoyer. Commutation de paquets : chaque paquet routé individuellement.
- IPv4 : RFC 760 (1980), norme actuelle RFC 791 (1981) ; v0 à v3 n'étaient que des brouillons. Sans connexion, **best effort**, ni fiabilité ni ordre garantis ; supporte DiffServ.
- Sablier : tout le monde parle IP ; liaison et couches hautes restent locales.
- Terminal (ne relaie pas), routeur (relaie), passerelle (entre deux domaines).

Routage (p. 7 et 8)

Routage : découvrir la topologie (protocole de routage). Relaiage : utiliser cette information pour choisir le saut suivant, paquet par paquet, chaque nœud de façon indépendante (hop by hop). Routage par la source (elle impose le chemin) : pas sur Internet, on ne peut pas lui faire confiance ; utilisé en réseaux de capteurs ad hoc.

Boucles : si l'information de topologie est fausse, un paquet peut tourner indéfiniment. Les nœuds ne mémorisent pas les paquets vus et les paquets ne stockent pas les nœuds visités. Solution : le **TTL**, compteur décrémenté à chaque nœud ; le paquet est détruit à 0.

Succès d'IPv4 (p. 17) : IPX et X.25 éliminés, remplacement global impossible ; IPv6 naît de l'épuisement des adresses.

Mobilité (hta2p p. 10) : l'adresse dépend de l'emplacement. Mobile IP : adresse « home » officielle, adresse locale dans le réseau visité, joignable à l'adresse home. **Allocation** (p. 9) : trop tardif, CIDR n'a pas rendu l'allocation géographique.

19. En-tête IPv4

Source : hta2-ipv4.pdf, p. 10 à 16

Bits	Champ	À retenir
4	Version	toujours 4
4	IHL	longueur d'en-tête en mots de 32 bits ; minimum 5 (20 octets)
6 + 2	DSCP, ECN	DSCP surtout en réseau privé ; ECN signale la congestion sans jeter, peu utilisé
16	Total Length	octets, en-tête compris : de 20 à 65 535
16	Identification	numéro du paquet, pour regrouper ses fragments
3	Flags	bit réservé, DF (ne pas fragmenter, jeter), MF (d'autres fragments suivent)
13	Fragment Offset	position dans le paquet d'origine, en unités de 8 octets
8	TTL	anti-boucle
8	Protocol	contenu : ICMP 1, IGMP 2, TCP 6, UDP 17 (liste IANA)
16	Header Checksum	en-tête seul ; inclut le TTL, donc recalculé par chaque routeur
32 + 32	Source, Destination	adresses IP
var.	Options	si IHL > 5 ; expérimentales, presque toutes dépréciées (RFC 6814)

0x45, premier octet de presque tous les paquets

0x45 = 0100 0101 : version 0100 = 4, IHL 0101 = 5, soit $5 \times 4 = 20$ octets d'en-tête, sans options. La slide p. 10 note que certaines implémentations codent 0x45 en dur.

Décoder l'en-tête de la capture Wireshark (hta3p-ipv4-tools.pdf, p. 22)

Les 20 octets IP, après les 14 octets Ethernet :

45 00 00 3c 6f 89 00 00 02 11 30 8d
c0 a8 64 0f ac 43 47 a0

45	version 4, IHL 5	20 octets
00	DSCP 0, ECN 0	
00 3c	Total Length	$3 \times 16 + 12 = 60$ octets
6f 89	Identification	28 553
00 00	Flags, Offset	DF = 0, MF = 0, offset 0
02	TTL	2
11	Protocol	$16 + 1 = 17$: UDP
30 8d	Checksum	0x308D
c0 a8 64 0f	Source	192.168.100.15
ac 43 47 a0	Destination	172.67.71.160

Un TTL de 2 et des ports UDP élevés : c'est un paquet de traceroute (les lignes voisines de la capture sont des « Time-to-live exceeded »). Wireshark affiche les mêmes valeurs.

Piège

IHL compte des mots de 4 octets, Total Length des octets, Fragment Offset des blocs de 8 octets. Trois unités différentes dans le même en-tête.

20. Fragmentation

Source : hta2-ipv4.pdf, p. 9 et 12 ; calcul : méthode générale

À retenir

- MTU : Ethernet 1500 octets, jumbo 9000 (non standard).
- On fragmente si le message dépasse la MTU ou si un lien du chemin a une MTU plus petite ; réassemblage au saut suivant ou au récepteur (question ouverte de la slide).
- Path MTU discovery (tester la MTU du chemin avant d'envoyer) : pas dans IPv4, selon la slide p. 9.
- DF à 1 : le routeur jette au lieu de fragmenter, et ICMP peut signaler « fragmentation needed » (type 3, hta3 p. 4).

Méthode de calcul (méthode générale)

1. Données du paquet d'origine : $D = \text{Total Length} - 20$ (sans options).
2. Données par fragment : $\lfloor (\text{MTU} - 20) / 8 \rfloor \times 8$, car l'offset compte en blocs de 8 octets ; seul le dernier fragment peut avoir une taille quelconque.
3. Offset d'un fragment = octets de données déjà envoyés / 8.
4. MF = 1 sur tous les fragments sauf le dernier ; même Identification pour tous.

Paquet de 4000 octets (20 d'en-tête) sur un lien de MTU 1500

$D = 3980$. Par fragment : $1500 - 20 = 1480$, multiple de 8 (185×8).

Fragment	Données	Total Length	Offset	MF
1	1480	1500	0	1
2	1480	1500	185	1
3	1020	1040	370	0

Contrôle : $1480 + 1480 + 1020 = 3980$; offset $185 = 1480 / 8$, $370 = 2960 / 8$.

Place de ce calcul à l'examen : les slides ne donnent pas d'exercice de fragmentation, **À VÉRIFIER**.

21. Plans et signalisation

Source : hta3-ipv4-protocols.pdf, p. 2 et 3

À retenir

- **Plan de données** : transporter les données utilisateur.
- **Plan de contrôle** : rendre le plan de données possible (établir et fermer une connexion, allouer des ressources QoS, acquitter, keepalive).
- **Plan de gestion** : rendre les deux autres possibles (routage, détection d'erreurs de lien, configuration). Définition jugée dépassée par la slide : aujourd'hui le routage est classé contrôle, la gestion se limite à la configuration à distance.
- ARP et DHCP sont « entre les couches » ; la slide laisse la question du plan ouverte.

Couche	Données	Contrôle	Gestion
Application	HTTP, RTP, DNS	RTCP, SIP	SNMP, OpenFlow
Transport	TCP, UDP	TCP	
Réseau	IPv4	ICMP	ICMP, RIP, OSPF
Liaison	Ethernet	LLC	STP

Signalisation : échanges de contrôle et de gestion qui ne sont pas des données utilisateur. *Dans la bande* : même canal que les données (« bonjour », « au revoir »). *Hors bande* : canal séparé (un signe de la main). LITU-T utilise souvent un canal dédié (fréquence ou tranche de temps) ; l'IETF envoie simplement des paquets IP.

22. ICMP

Source : hta3-ipv4-protocols.pdf, p. 4 à 7

À retenir

- Internet Control Message Protocol, RFC 792. Messages d'erreur (acquittements négatifs pour des conditions précises) et requêtes d'information, car IP est sans connexion et best effort.
- Encapsulé directement dans IP, **Protocol = 1**. Dans la bande pour IP hors bande pour TCP et UDP
- En-tête : **Type** (8 bits) et **Code** (8 bits, sous-type), plus une somme de contrôle, car IP ne contrôle pas sa charge utile.

Type	Message	Sous-types, usage
0	Echo Reply	réponse au ping
3	Destination Unreachable	net, host, protocol, port unreachable ; fragmentation needed ; net, host unreachable for ToS
5	Redirect	un routeur signale une meilleure route à la source (configuration manuelle)
8	Echo Request	ping
9, 10	Router Advertisement, Solicitation	découverte des routeurs locaux ; remplacé par DHCP, étendu par IPv6 Neighbor Discovery
11	Time Exceeded	TTL expiré ; délai de réassemblage dépassé

Piège

Ordre des types 9 et 10 : la slide écrit « Router Solicitation / Router Advertisement (type=9, 10) ». Selon la RFC 1256, 9 est l'Advertisement et 10 la Solicitation ; le tableau suit la RFC. Point à confirmer : **À VÉRIFIER.**

Ping et traceroute (p. 6)

Ping : Echo Request, attendre l'Echo Reply ; mesure latence et taux de perte.

Traceroute : envoyer avec TTL = 1, 2, 3... ; le routeur où le TTL tombe à 0 renvoie un Time Exceeded, ce qui révèle chaque saut.

Sécurité (p. 7)

- Aucune authentification : qui intercepte mon trafic peut m'envoyer un faux Destination Unreachable ou Redirect.
- Déni de service : ping flood, saturation du tampon de réassemblage.
- Les messages d'erreur renvoient le paquet fautif (au moins 8 octets, jusqu'à la MTU) ; Echo peut transporter n'importe quelle quantité de données, d'où le tunnel ping pour contourner des restrictions.
- Cartographie de réseaux distants à la recherche de machines vulnérables : les pare-feux filtrent ICMP (pas d'Echo Reply, Unreachable venant toujours de la passerelle), parfois totalement.

23. DHCP

Source : [hta3-ipv4-protocols.pdf](#), p. 8 à 10 ; [hta3p-ipv4-tools.pdf](#), p. 14, 15 et 24 ;
[ressources/demos/dhcp/](#)

À retenir

- Dynamic Host Configuration Protocol, RFC 2131. Le serveur attribue les adresses aux hôtes du réseau local.
- Techniquement de la couche application, sur **UDP** : serveur port **67**, client port **68**.
- Options TLV : masque, passerelle, DNS, serveur de temps, nom d'hôte, image de boot.
- **État mou** : le bail (lease) doit être renouvelé, sinon il expire ; utiliser une adresse expirée pose problème si le serveur l'a donnée à un autre hôte.

Échange en quatre temps, **DORA** :

1. **Discover**, client vers tous : « qui est le serveur ? », broadcast liaison et IP.
2. **Offer**, serveur vers client : une adresse proposée (plusieurs serveurs possibles).
3. **Request**, en broadcast : « je prends celle-ci », tous les serveurs sont prévenus.
4. **Ack**, serveur vers client : confirmation.

Autres messages : decline, renew, release. (La p. 9 montre la RFC 2322, un « serveur DHCP » en pinces à linge.)

Outils et démo (hta3p)

Client `dhclient`, serveur Kea, tous deux de l'ISC. Démo Mininet : deux hôtes `na` et `nb` reliés par un lien (`ethAB` et `ethBA`) ; `na` reçoit `10.0.1.1/24` et lance `kea-dhcp4 -c kea.conf`, `nb` lance `dhclient -d ethBA`.

Lire `kea.conf` de la démo

Sous-réseau `10.0.1.0/24`, pool `10.0.1.10` à `10.0.1.30` : $30 - 10 + 1 = 21$ adresses. Passerelle (routers) `10.0.1.1`, l'adresse de `na`. Bail 20, renou-

vement 10 : secondes selon la documentation de Kea (absent des supports). Extrait du fichier :

```
"Dhcp4": {
  "valid-lifetime": 20,
  "renew-timer": 10,
  "rebind-timer": 10,
  "interfaces-config": { "interfaces": [ "ethAB" ] },
  "subnet4": [ {
    "id": 1,
    "subnet": "10.0.1.0/24",
    "pools": [ { "pool": "10.0.1.10 - 10.0.1.30" } ],
    "option-data": [
      { "name": "routers", "data": "10.0.1.1" } ]
  } ]
}
```

C'est du JSON : un bon support pour s'entraîner à la conversion en YAML (concept 5).

24. ARP

Source : hta3-ipv4-protocols.pdf, p. 2; capture hta3p-ipv4-tools.pdf, p. 23

Ce que disent les slides. ARP est cité comme protocole « entre les couches », avec DHCP (hta3 p. 2). La capture tcpdump de hta3p p. 23 en montre un échange :

```
ARP, Request who-has 10.1.33.2 tell 10.1.33.1, length 28
ARP, Reply 10.1.33.2 is-at 24:5e:be:7b:be:c7, length 46
```

Aucune slide n'en décrit le fonctionnement. La fiche 2023 annonçait un TD sur ARP.

Fonctionnement standard (RFC 826, hors slides). Pour envoyer un paquet IP sur Ethernet, il faut l'adresse MAC du saut suivant : la destination si elle est dans mon sous-réseau, la passerelle sinon.

1. Consulter le cache ARP
2. Sinon, diffuser en broadcast Ethernet une requête « qui a 10.1.33.2 ? dites-le à 10.1.33.1 ».
3. Le propriétaire répond en unicast avec sa MAC ; l'émetteur la met en cache (entrée à durée limitée).

ARP ne s'authentifie pas : une fausse réponse détourne le trafic (empoisonnement de cache). Ce point recoupe les modules ARP de CyberOps (BMEVI-HIAV43).

Profondeur attendue à l'examen : **À VÉRIFIER.**

25. DNS

Source : hta3-ipv4-protocols.pdf, p. 11 à 18

À retenir

- Noms vers adresses IP, via `getaddrinfo`. Outils `host`, `nslookup`, `dig` ; serveurs BIND 9, Unbound, Dnsmasq.
- Ancêtre : `/etc/hosts`. RFC 882 et 883 (1983), puis RFC 1034 et 1035 (1987).
- Espace de noms en arbre. URL : `scheme://location/path/file` ; le DNS résout la partie `location`. URI : désignation plus générale.
- TLD : `.com`, `.org`, `.edu`, `.gov`, `.net`, ccTLD (`.hu`, `.co.uk`). FQDN : nom complet.
- Enregistrements : **A** (IPv4), **AAAA** (IPv6), **MX** (serveur de courrier), et d'autres.

Résolution (p. 14 et 15)

Base distribuée : chaque serveur tient un sous-arbre, par exemple tout ce qui finit par `.tmit.bme.hu`, et connaît certains serveurs au-dessus de lui ; cette topologie est indépendante du réseau. Le client connaît un résolveur local (souvent donné par DHCP) et demande les types voulus (A et AAAA). La requête monte puis redescend l'arbre jusqu'au serveur **autoritaire** ou jusqu'à une réponse en cache ; le client peut exiger une réponse autoritaire. Résolution inverse : d'une adresse IP vers un nom.

Organisation, transport, sécurité (p. 16 à 18)

- ICANN : espace d'adresses, numéros de protocole, TLD, serveurs racine. ISOC : `.org`. Registrars accrédités ; propriété vérifiable par WHOIS (RFC 3912).
- Transport : UDP/53 à l'origine, TCP/53 (messages longs, fiabilité, 1989), TLS (pas partout), HTTPS (sûr mais contourne le mécanisme de résolution d'origine).
- Faiblesses : ni chiffrement ni authentification, empoisonnement de cache, noms ressemblants (`facebock.com`, autre TLD, caractères grecs).

26. Outils de configuration réseau

Source : hta3p-ipv4-tools.pdf, p. 2 à 17

Windows : dialogue de paramètres réseau, `ipconfig` (état, client DHCP) ; côté serveur Active Directory (authentification, DHCP, DNS), IIS, RDS. Linux : noyau et outils GNU assemblés en distributions, installation par gestionnaire de paquets ; incontournable en réseau.

Commande (exemples des slides)	Rôle
<code>ifconfig eth0 10.0.3.14/24 up</code>	ajouter une adresse, activer; <code>ifconfig eth0 down</code>
<code>route -n</code>	lister la table de relayage (numérique)
<code>route add 10.0.20.0/16 dev eth0</code>	ajouter une route
<code>netstat -pnaut</code>	connexions ouvertes : processus, numérique, tous états, UDP, TCP
<code>netstat -r, -i, -s</code>	routes, statistiques d'interfaces, de protocoles
<code>ip addr add 10.0.20.30/24 dev eth0</code>	iproute2 (Linux) : ajouter une adresse
<code>ip link set dev eth0 up</code>	activer une interface
<code>ip route add 10.20.30.0/24 dev eth0 via 10.0.20.10</code>	route via une passerelle

`ip` (`iproute2`) remplace `ifconfig` et `route` sur Linux ; ces deux-là existent sur tous les Unix, avec des options qui varient.

Piège

Dans `route add 10.0.20.0/16`, `10.0.20.0` n'est pas une adresse de réseau /16 : le réseau /16 qui la contient est `10.0.0.0/16`. À l'examen, toujours vérifier qu'une route ou un réseau donné est aligné sur son préfixe.

Configuration permanente (p. 15 et 16)

```
# /etc/network/interfaces
auto eth0
auto eth1
iface eth0 inet dhcp
iface eth1 inet static
```

```
address 10.0.1.11/24
gateway 10.0.1.1
```

Le fichier `/etc/resolv.conf` contient la configuration DNS; `dhclient` le gère automatiquement. Gestionnaires de réseau :

- NetworkManager, configuré avec `nmcli` ou une interface graphique ;
- `systemd-networkd`, règles dans `/etc/systemd/network`, avec son propre client DHCP ;
- `netplan` d'Ubuntu, règles dans `/etc/netplan/`, au-dessus des deux précédents.

Shell (p. 6 à 8) : interpréteur programmable (variables, `if`, `while`, `for`, fonctions), redirections `>` `<` et tubes `|`, tâches de fond. Commandes courantes : `man`, `cat`, `less`, `ls`, `find`, `sort`, `sed`, `ps`, `top`, `kill`, `sudo`, `lspci`...

27. Outils de diagnostic et de capture

Source : hta3p-ipv4-tools.pdf, p. 17 à 24

À retenir

- **ping** : Echo Request, la cible répond par Echo Reply. Un ping réussi ne garantit pas que le reste du trafic passe.
- **traceroute** : TTL croissant, on regarde qui renvoie Time Exceeded. Par défaut des paquets UDP vers un port aléatoire ; -I ICMP Echo Request ; -T TCP SYN. Le type de paquet compte, car les pare-feux ne les traitent pas pareil.
- **tcpdump** : enregistre et décode le trafic, sauvegarde dans un fichier (rejouable avec tcpreplay), mode promiscuous pour voir le trafic destiné à d'autres.
- **Wireshark** : équivalent graphique (ancien nom Ethereal, 2006) ; **tshark** en console, plus riche que tcpdump.
- **Mininet** : bac à sable de VM légères et liens virtuels, piloté en Python.

Lire la sortie de ping (p. 18)

```
64 bytes from ... (152.66.121.21): icmp_seq=1 ttl=60 time=1.43 ms
24 packets transmitted, 24 received, 0% packet loss
rtt min/avg/max/mdev = 1.062/1.282/1.443/0.097 ms
```

Par ligne : séquence, TTL restant, aller-retour. Bilan : aucune perte, RTT moyen 1,282 ms.

Lire traceroute (p. 19)

Chaque ligne numérotée est un saut, avec trois mesures de temps. Premier saut vers telex.hu : _gateway (192.168.100.254), la passerelle par défaut. Vers isc.org en mode UDP, les sauts 9 à 30 affichent * * * : aucune réponse (filtrage) et la destination n'est jamais atteinte ; avec -I ou -T, elle répond au 10^e saut.

Capture tcpdump (p. 23) : ttl 64, id 15149, offset 0, flags [DF], proto ICMP (1), length 84, les champs de l'en-tête IPv4.

28. Méthode d'examen

Source : types d'exercices : hta2p-ipv4-practical.pdf, p. 11 à 14; hta1p-protocols.pdf, p. 26

Format de l'épreuve : **À VÉRIFIER**. Modèles donnés par les slides : les quatre « Practical Examples » de hta2p et la conversion XML vers JSON ou YAML.

À retenir

1. Validité : octets ≤ 255 , masque continu, réseau aligné.

2. Convertir le masque en /n (ou l'inverse).

3. Repérer l'octet intéressant et le nombre magique $256 - \text{octet}$.

4. Écrire en binaire l'octet intéressant, aligné avec le masque.

5. Contrôler : réseau multiple de m , broadcast = réseau $+m - 1$, hôtes $= 2^{32-n} - 2$.

Question	Méthode
masque et préfixe	concept 10, compter les 1
réseau, broadcast, hôtes	ET bit à bit ou nombre magique (11, 12)
même sous-réseau ?	comparer les n premiers bits (13)
découper, VLSM	plus grands besoins d'abord, alignement (14, 15)
fusionner des routes	seul le dernier bit diffère, même saut suivant (16)
interface de sortie	tous les préfixes qui correspondent, puis le plus long (17)
décoder un en-tête	0x45, unités des champs (19)
XML vers JSON, YAML	règles du concept 5

Piège

• Oublier le « -2 » des hôtes, confondre dernier hôte et broadcast.

• Fusionner des routes non alignées ou vers des interfaces différentes.

• S'arrêter à la première route qui correspond au lieu de la plus longue.

29. Formulaire

À retenir

Poids d'un octet	128, 64, 32, 16, 8, 4, 2, 1
Octets de masque	128, 192, 224, 240, 248, 252, 254, 255
Adresses d'un /n	2^{32-n}
Hôtes d'un /n	$2^{32-n} - 2$
Nombre magique	256 moins l'octet du masque
Réseau	adresse ET masque
Broadcast	partie hôte à 1
s bits empruntés	2^s sous-réseaux, préfixe $n + s$
Sous-réseau k	base $+k \times 2^{32-(n+s)}$
Fusion de deux /n	même préfixe sauf le dernier bit, même saut suivant : un $/(n - 1)$
Routage	plus long préfixe ; 0.0.0.0/0 en dernier recours

Adresses	
Classes A, B, C	1 à 126 (/8), 128 à 191 (/16), 192 à 223 (/24)
D, E	multicast 1110, réservé 11110
Privées	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
Loopback	127.0.0.1 (réseau 127)
En-tête IPv4	
Version, IHL	4 ; IHL $\times$ 4 octets, minimum 20
Total Length	20 à 65 535 octets
Fragment Offset	unités de 8 octets ; flags DF, MF
Protocol	ICMP 1, IGMP 2, TCP 6, UDP 17
MTU	Ethernet 1500 ; jumbo 9000
Protocoles	
ICMP	0 Echo Reply, 3 Unreachable, 5 Redirect, 8 Echo Request, 11 Time Exceeded
DHCP	UDP 67 (serveur), 68 (client) ; DORA ; bail
DNS	UDP/53, TCP/53 ; A, AAAA, MX
PCM	8 kHz $\times$ 8 bits = 64 kbit/s
Token, leaky bucket	K = moyen ; B = (max – moy) $\times$ rafale